



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

状态模式（State）

一、意图（Intent）

允许一个对象在其内部状态改变时改变它的行为，使其看起来像修改了所属类别。

站在嵌入式 C 的角度，可以把这句话改写成：

把“当前处于什么状态”与“当前应该怎么响应事件”绑定起来。

状态模式真正控制的是：

- 当前状态的归属位置
- 状态切换的合法路径
- 不同状态下对同一事件的不同处理方式
- entry / run / exit 的执行时机

一句话记住它：

不是 if-else 多，而是“行为由当前状态决定”。

二、动机（Motivation）

嵌入式系统里有大量天然的状态机对象：

- 设备初始化流程
- 蓝牙 / Wi-Fi / TCP 连接过程

- 充电状态
- 电机控制阶段
- UI 页面流程
- OTA 升级过程

最常见的坏味道是：

```
if (ctx->state == INIT) { ... }  
else if (ctx->state == IDLE) { ... }  
else if (ctx->state == RUNNING) { ... }  
else if (ctx->state == ERROR) { ... }
```

然后在多个函数里都重复判断：

- 收到命令时判断一次
- 定时器到时判断一次
- 中断事件来时判断一次
- 超时恢复时再判断一次

这样会带来几个问题：

1. 状态判断到处散落

状态本来应该集中管理，结果变成整个系统都在问 `ctx->state`。

2. 切换规则不集中

从哪个状态能切到哪个状态、切换时要做什么，容易散落到多个函数里。

3. entry / exit 动作容易漏

很多硬件相关逻辑不是“改个枚举值”就结束，而是要：

- 进入状态时打开资源
- 退出状态时释放资源
- 切换时重置计数器 / 超时器 / 标志位

4. 状态与行为没有绑定

同一个事件在不同状态下处理方式不同，但代码组织却没有体现这一点。

所以状态模式在嵌入式里最重要的价值是：

把状态、事件处理、切换逻辑和出入状态动作收口到统一结构里。

三、适用性 (Applicability)

适合考虑状态模式的条件有：

- 模块有明确的有限状态

- 同一事件在不同状态下**处理不同**
- 状态切换有严格规则
- 进入和退出状态时要执行动作
- 业务层不希望到处直接判断状态值

典型场景：

- 通信连接管理
- 设备生命周期管理
- 任务流程控制
- 充电 / 休眠 / 唤醒
- 升级与恢复流程

四、结构（Structure）

```
应用层事件
  |
  v
Context（状态上下文） ----> Current State
                                |
                                +-- entry()
                                +-- run()/handle_event()
                                +-- exit()
```

从嵌入式执行路径看，可以理解为：

```
事件 / 定时器 / 轮询
  |
  v
状态机上下文 ctx
  |
  v
当前状态处理函数
  |
  +-- 保持当前状态
  +-- 切换到新状态
  +-- 终止 / 错误 / 上报
```

五、参与者（Participants）

1. Context（状态上下文）

它保存：

- 当前状态
- 状态机公共数据

- 与状态切换相关的计数器、标志位、资源句柄

在嵌入式 C 里，它通常就是一个上下文结构体。

2. State（抽象状态）

定义统一的状态处理接口。

如果做得更工程化，通常会拆成：

- `entry`
- `run` 或 `handle_event`
- `exit`

3. ConcreteState（具体状态）

每个具体状态负责：

- 本状态下如何处理事件
- 何时跳转到新状态
- 进入和退出时做什么动作

六、协作（Collaborations）

通常的执行过程是：

1. 初始化状态机上下文
2. 设定初始状态
3. 事件或调度到来时，交给当前状态处理
4. 当前状态决定：
 - 自己处理完
 - 向上层传播
 - 切换到新状态
5. 若切换，则执行 `exit -> transition -> entry`

状态模式的关键不是“有状态枚举”，而是：

当前状态自己决定当前事件如何被解释。

七、效果（Consequences）

收益

1. 状态逻辑集中

每个状态的行为和切换规则更容易放在一起看。

2. 减少散落的条件分支

不用在每个业务函数里都反复问“当前 state 是什么”。

3. 更适合复杂生命周期管理

尤其适合初始化、连接、异常、恢复这类有阶段边界的流程。

4. entry / exit 语义清晰

资源申请、资源释放、计时器重置、日志上报，都能放到明确位置。

代价

1. 状态数多时结构会变大

状态越细，函数与表项越多。

2. 需要更严格的切换规则设计

否则状态会爆炸，或者出现非法跳转。

3. 调试需要观察状态轨迹

单看某个函数常常不够，要看“从哪里来、往哪里去”。

八、实现要点 (Implementation)

1. 状态上下文要集中

不要把状态相关数据散落到多个全局变量里。

常见做法是：

- 一个 `ctx`
- 里面包含当前状态
- 再加共享标志位、计时器、错误码等

2. 区分“状态”和“事件”

状态是系统当前阶段，事件是触发处理的输入。

两者不能混成一个大枚举。

3. entry / exit 中放资源边界动作

例如：

- 打开 / 关闭外设
- 启停超时计时器
- 清空或冻结缓冲区
- 设置 busy / ready / error 标志

4. 非法跳转要显式防御

嵌入式里状态机常涉及硬件时序，非法跳转不能默默放过。

5. 层级状态机要谨慎使用

它很强，但如果团队还在频繁改业务流程，先把平面状态机理顺更重要。

九、典型应用

1. 连接管理

例如：

- DISCONNECTED
- CONNECTING
- CONNECTED
- RETRY_BACKOFF
- ERROR

相同的“收到数据”“连接超时”“用户断开”事件，在不同状态下意义完全不同。

2. 电源 / 充电 / 升级流程

这类流程都有强状态边界和明显的 entry / exit 动作，非常适合状态模式。

总结

状态模式在嵌入式 C 里，本质上就是把“当前阶段”和“当前处理规则”绑在一起，让行为随状态切换而切换。